Unit1

Unit 1. Working with audio data

> In this unit, you will gain an understanding of the fundamental terminology related to audio data, including waveform, sampling rate, and spectrogram. You will also learn how to work with audio datasets, including loading and preprocessing audio data, and how to stream large datasets efficiently.

波形/采样率/频谱图/加载预训练音频数据/大数据流式加载

- **声音数字化**：自然界的声音是一段连续的声波（模拟信号），但计算机和数字设备只能处理有限的数字。因此，我们需要通过麦克风捕捉声音，然后用“模数转换器 (ADC)”将其变成数字信号。
- 使用**Sinc 函数插值（Whittaker-Shannon 插值公式）** 还原

Introduction to audio data

Sampling and sampling rate

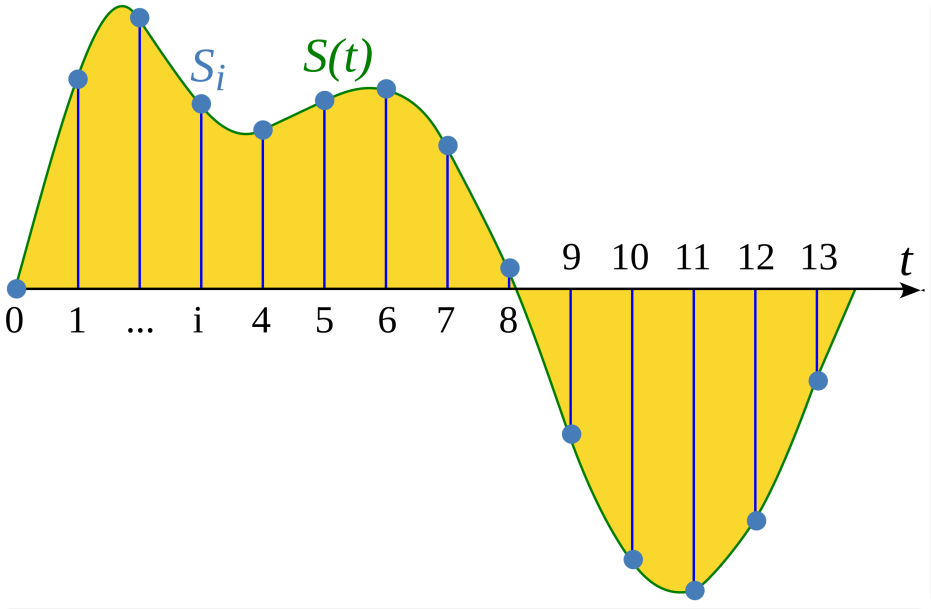

image-159.png

- **Sampling, Sampling Rate**：如图，顾名思义
- **奈奎斯特极限 (Nyquist Limit)：** 你能捕捉到的最高声音频率，刚好是采样率的一半。人说话的频率通常在 8 kHz 以下，所以用 16 kHz 来采样人类语音是完全足够的。
  - $f_{\max} = \frac{f_s}{2}$ （其中 f_s 是采样率），采样率要高声音曲线越平滑
- **Resampling**: audio data 预训练的一种，将不同的采样率统一起来的过程

Amplitude and Bit Depth

- **Amplitude**：代表声音的声压级，我们耳朵感知到的就是**音量（响度）**
- **Bit Depth**：决定记录 amplitude 的精确程度，由于记录时精确不够需要四舍五入，这部分值则为噪声，精度越高噪声越小。
  - **16-bit 音频：** 有 16 个二进制位，能组合出 $2^{16} = 65536$ 个阶

The Frequency Spectrum（频谱图）

- 某一个时刻，一个声音到底是由哪些 频率 组成的，就需要用到频谱图
  - 这个算法没太懂！！！！

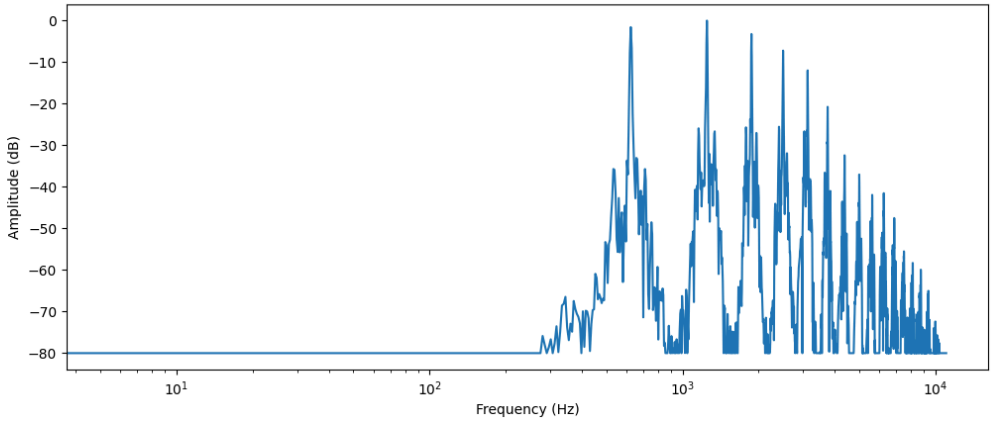

image-160.png

- **DFT (Discrete Fourier Transform，离散傅里叶变换)：** 自然界的声波是连续的，但计算机里存的音频是离散的采样点（也就是数组）。DFT 就是专门用来处理这种**离散数组**的傅里叶变换公式。给定一段长度为 N 的音频数组，DFT 可以计算出它包含的频率成分。
- **FFT (Fast Fourier Transform，快速傅里叶变换)：** 计算机在计算原始的 DFT 时，计算量非常大。如果要处理 N 个采样点，它的时间复杂度是 $O(N^2)$。对于动辄几万采样率的音频来说，这个计算慢得让人无法接受。FFT 并不是一种新的变换，而是计算 DFT 的一种**极其高效的算法**。

Spectrogram

- **语谱图**：通过**短时傅里叶变换 (STFT)** 将一段长音频切成无数个几毫秒的小段，然后把它们各自的频谱图按时间顺序拼接起来，达到同时看到**时间、频率和音量**

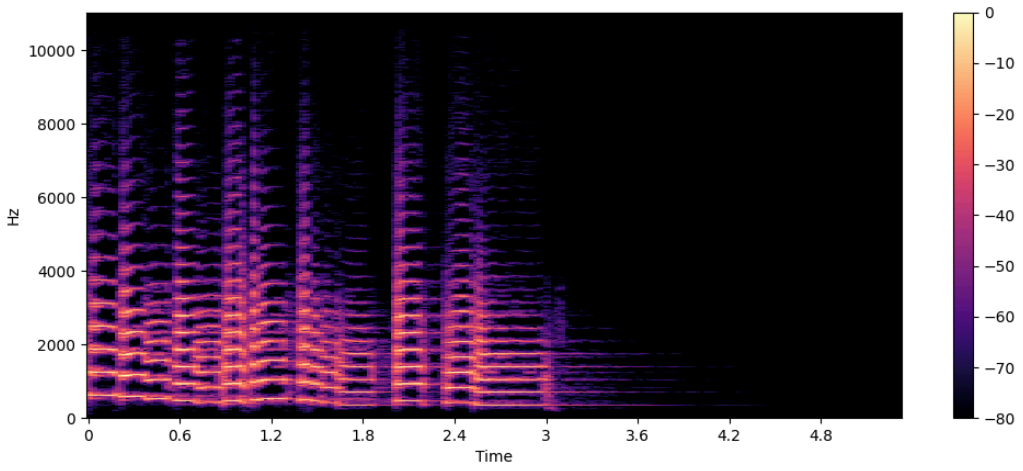

image-161.png

Mel spectrogram

- 普通的语谱图，频率的刻度是平均分布的（线性的）。但是，**人类的耳朵对低音的细微变化非常敏感，而对高音的辨识度很差**。
- 为了让 AI 像人耳一样去感知声音，设计了“梅尔刻度 ”。通过一组特殊的滤波器，把普通语谱图转换成了更符合人类听觉特性的**梅尔语谱图**。

Load and explore an audio dataset

数据集特征

```
Dataset({
    features: ['path', 'audio', 'transcription', 'english_transcription',
'intent_class', 'lang_id'],
    num_rows: 654
})
```

- 展示了该数据集的文件数与基本特征——即包含 654 个音频文件，每个文件都配有文字转录、英文翻译以及标注用户查询意图的标签

raw auto data 特征

```
{'path': 'en-AU~PAY_BILL/response_4.wav',
 'audio': <datasets.features._torchcodec.AudioDecoder at 0x7bee1d46a630>,
 'transcription': 'I would like to pay my electricity bill using my card can you
please assist',
 'english_transcription': 'I would like to pay my electricity bill using my card
can you please assist',
 'intent_class': 13,
 'lang_id': 2}
```

视觉听觉可视化

```python
import gradio as gr
import librosa
import matplotlib.pyplot as plt
import librosa.display

# --- 视觉部分：画出音频波形图 ---
array = example["audio"]["array"]
sampling_rate = example["audio"]["sampling_rate"]

plt.figure(figsize=(12, 4))
plt.title("Audio Waveform of Example 0")
librosa.display.waveshow(array, sr=sampling_rate)
plt.show()

# --- 听觉部分：构建一个随机播放的迷你播放器 ---
def generate_audio():
    random_example = minds.shuffle()[0]
    audio = random_example["audio"]
    return (
        audio["sampling_rate"],
        audio["array"],
    ), id2label(random_example["intent_class"])

with gr.Blocks() as demo:
    gr.Markdown("MINDS-14 随机音频")
    with gr.Column():
        for _ in range(4):
            audio_data, label = generate_audio()
            gr.Audio(audio_data, label=label)

# 启动播放器
demo.launch(debug=True)
```

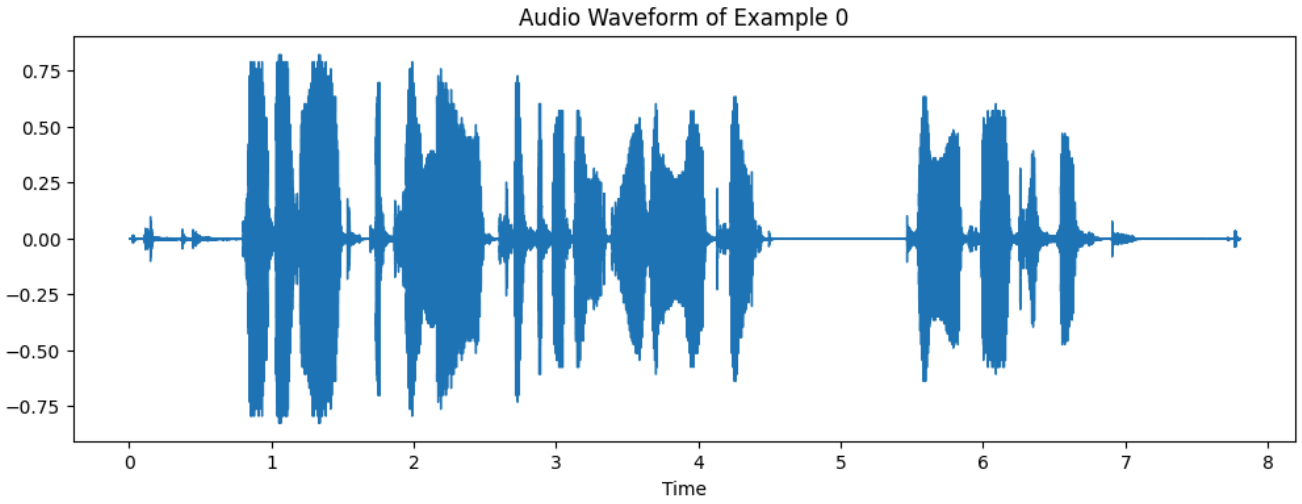

image-162.png

Preprocessing audio data

Resampling

每个音频数据集在录制时都有自己的采样率（如 MINDS-14 是 8 kHz）。但大多数预训练模型（如 Whisper, Wav2Vec2）是在 16 kHz 上训练的。

- **如果不对齐**：模型会把原本慢速的声音听成“快进”或“变调”的声音，导致识别完全失败。

背后原理：奈奎斯特定理

- **上采样 (Upsampling)**：从 8k 到 16k。原理是在现有的采样点之间，根据波形的平滑曲线“插值”计算出新的点。
- **下采样 (Downsampling)**：直接删除一半的点会产生**混叠现象 (Aliasing)**——高音并不会自动消失，而是会变成低频噪音。必须先用低通滤波器滤掉高频信号，再取点。
  - ❓ 这里下采样写一个可视化直观理解一下

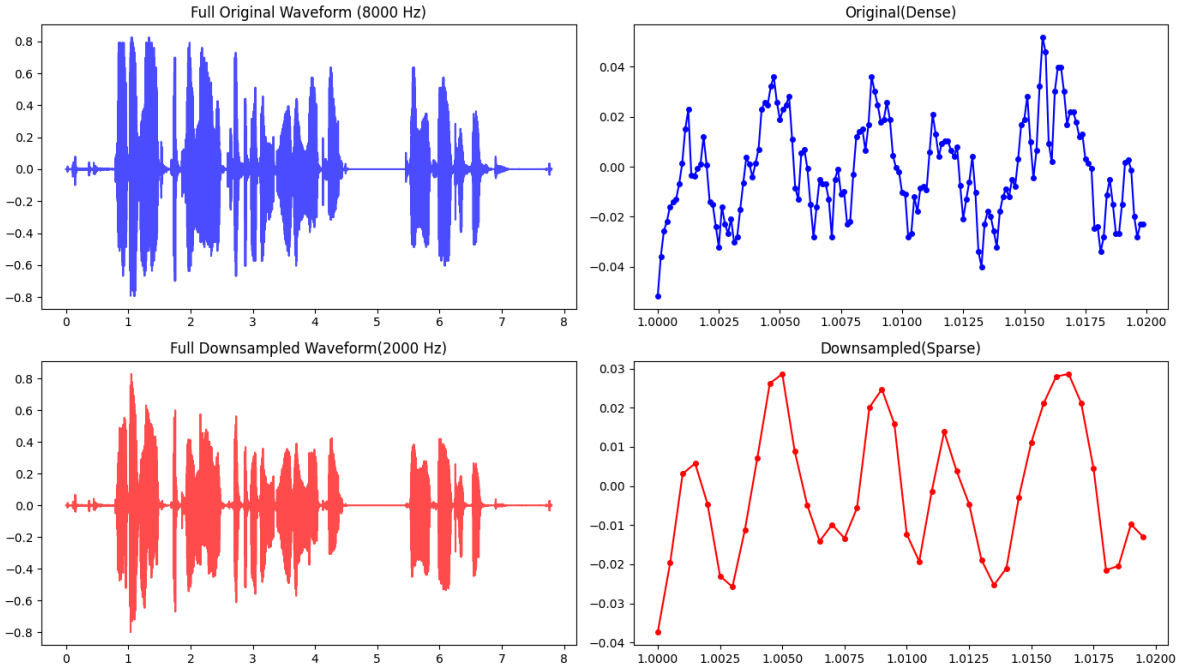

image-163.png

Filtering

- 由于**OOM**，要选择更适合训练的 auto data，所以引入了 `filter` 方法进行过滤。
  - ℹ 在 16 kHz 采样率下，30 秒的音频包含 $16,000 \times 30 = 480,000$ 个数据点。如果基于 Transformer 架构, 复杂度为 $O(N^2)$。当 N 达到几十万时，注意力矩阵的大小会瞬间撑爆显存
- 通常根据长度、空白（VAD）、噪音（SNR）进行过滤

Model-Specific Preprocessing

- 为了更容易提取语音特征，我们通常需要把音频转换成 **Mel Spectrogram** 等特征矩阵

Streaming audio data

- 解决内存不够的问题，设计了 **streaming mode** ——遍历过程中抓取样本，处理完即丢弃
- 需要使用python中的 `iterator` 来调用样本